PR Rechnernetze und Netzwerkprogrammierung 621.801/621.802

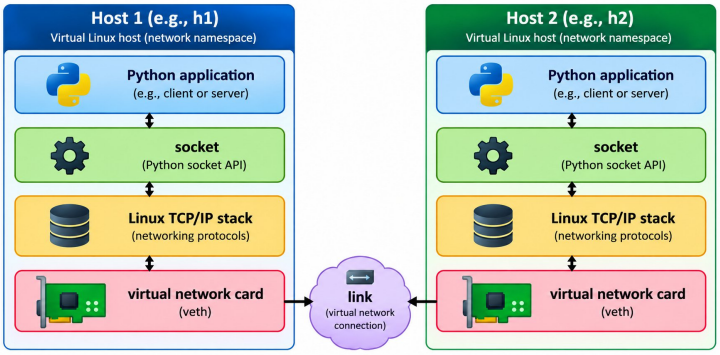
Institut für Informationstechnologie (ITEC) · WS 2026/2027

Übungsblatt 04

Netzwerkprogrammierung mit Python und Mininet

Die Mininet-Laborumgebung

Bis zu diesem Übungsblatt wurden Programme direkt auf Ihrem Rechner ausgeführt. Nun soll ein Programm auf einem Rechner mit einer bestimmten IP-Adresse mit einem Programm auf einem anderen Rechner kommunizieren. Für die Übungen steht dafür eine virtuelle Netzwerkumgebung zur Verfügung.



Mininet erzeugt diese Netzwerkumgebung auf einem einzigen Linux-System. Die virtuellen Hosts besitzen eigene Netzwerk-Namespaces und eigene Interfaces. Dadurch können auf einem Laptop mehrere voneinander getrennte Netzwerkendpunkte betrieben werden.

Mininet stellt insbesondere folgende Komponenten bereit:

- Hosts: virtuelle Linux-Endsysteme, in denen normale Programme ausgeführt werden können.
- Switches: virtuelle Layer-2-Switches.
- Links: virtuelle Netzwerkverbindungen zwischen Hosts, Switches und gegebenenfalls Routern.
- Netzwerk-Namespaces: sorgen dafür, dass Interfaces, Routingtabellen und Prozesse eines virtuellen Hosts voneinander getrennt sind.

Ein wichtiger Unterschied zu einer Netzwerksimulation besteht darin, dass die Python-Anwendung nicht innerhalb eines simulierten TCP/IP-Protokolls läuft. Ein gewöhnliches Python-Programm mit socket.socket() kommuniziert mit dem tatsächlichen Linux-TCP/IP-Stack.

1. Starten der Mininet-VM

Für die Übungen wird eine vorbereitete virtuelle Maschine verwendet. Die VM enthält die für das Praktikum benötigten Werkzeuge, insbesondere Python 3, Mininet, Wireshark, tcpdump und weitere Linux-Netzwerkwerkzeuge.

Starten Sie zunächst die bereitgestellte VM und öffnen Sie ein Terminal. Prüfen Sie anschließend, ob Mininet verfügbar ist:

```
sudo mn --version
python3 --version
```

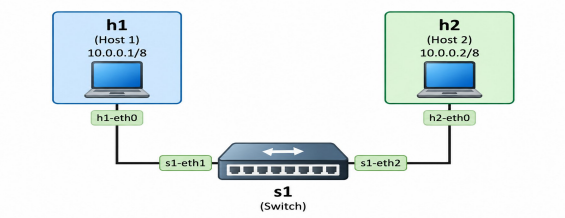
Falls die Versionen nicht angezeigt werden oder Mininet nicht gestartet werden kann, beheben Sie dies nicht eigenständig durch Änderungen an der Systeminstallation. Verwenden Sie die bereitgestellte VM und wenden Sie sich bei Problemen mit der Laborumgebung an die Übungsleitung.

2. Die Standardtopologie von Mininet

Starten Sie zunächst die Standardtopologie von Mininet:

```
sudo mn
```

Die Standardkonfiguration besteht aus zwei Hosts und einem Switch:



Untersuchen Sie die erzeugte Umgebung mit den folgenden Befehlen:

```
mininet> nodes
mininet> net
mininet> links
mininet> dump
```

Erklären Sie für jeden Befehl, welche Information ausgegeben wird.

Beenden Sie Mininet anschließend mit:

```
mininet> exit
```

Mininet sollte danach die erzeugten virtuellen Interfaces und Prozesse wieder entfernen.

Ü 4.1 Mininet-Hosts untersuchen

Starten Sie die Standardtopologie erneut. Befehle können über die Mininet-Shell gezielt in einem Host ausgeführt werden, indem dem Befehl der Hostname vorangestellt wird.

```
mininet> h1 ip addr
mininet> h2 ip addr
mininet> h1 ip route
mininet> h2 ip route
```

Beantworten Sie folgende Fragen:

- Welche Netzwerkinterfaces besitzt h1?
- Welche IPv4-Adresse besitzt h1?
- Welche IPv4-Adresse besitzt h2?
- Welche *path* ist auf h1 eingetragen?
- Was unterscheidet die Netzwerkschnittstelle eines Mininet-Hosts von einer normalen Netzwerkschnittstelle auf Ihrem Rechner?

Führen Sie anschließend einen Prozess innerhalb von h1 aus:

```
mininet> h1 hostname
mininet> h1 python3 --version
mininet> h1 pwd
```

Die Befehle werden innerhalb der Netzwerkumgebung von h1 ausgeführt. Überlegen Sie, warum ein in h1 gestartetes Programm nicht automatisch die Interfaces von h2 sehen kann.

Ü 4.2 Separate Konsolen für virtuelle Hosts

Für Client-/Server-Anwendungen ist es praktisch, mehrere Terminals gleichzeitig zu verwenden. In der vorbereiteten VM kann dafür xterm verwendet werden.

```
mininet> xterm h1 h2
```

Es sollten zwei neue Terminalfenster erscheinen. Jedes Fenster befindet sich im Netzwerk-Namespace des jeweiligen Mininet-Hosts. Das Terminal von h1 gehört daher zum virtuellen Host h1, das Terminal von h2 zum virtuellen Host h2.

Führen Sie in den beiden Terminals jeweils folgende Befehle aus:

```
ip addr
ip route
hostname
```

Erklären Sie die Ausgaben, welche Werte unterscheiden sich?

Ü 4.3 Eine Python-Anwendung in Mininet ausführen

Nun wird erstmals ein selbst geschriebenes Python-Programm innerhalb eines Mininet-Hosts ausgeführt. Der Server soll auf h2 laufen, während der Client auf h1 ausgeführt wird.
Verwenden Sie zunächst den folgenden UDP-Server:

```
import socket
server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind(("10.0.0.2", 5000))
print("Server listening on 10.0.0.2:5000")
while True:
    data, address = server.recvfrom(4096)
    print(f"Received from {address}: {data!r}")
```

Der zugehörige Client lautet:

```
import socket
client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
client.sendto(
    b"Hello from Mininet!",
    ("10.0.0.2", 5000))
```

Speichern Sie die beiden Programme als server.py und client.py. Starten Sie anschließend den Server im Terminal von h2 und den Client im Terminal von h1.
Beobachten Sie die Ausgabe des Servers. Ändern Sie die Nachricht des Clients und führen Sie den Client mehrfach aus.

Beantworten Sie:

- Welche IP-Adresse verwendet der Server?
- Warum darf der Client nicht einfach 127.0.0.1 (localhost) als Zieladresse verwenden?
- Welche Funktion hat die Portnummer 5000?
- Warum wird im Beispiel b"Hello from Mininet!" und nicht "Hello from Mininet!" verwendet?

Ü 4.4 Die Kommunikation mit tcpdump untersuchen

Die Anwendung soll nun nicht nur funktionieren, sondern auch auf Netzwerkebene untersucht werden.
Starten Sie den UDP-Server auf h2 und führen Sie auf h1 den Client aus.
Öffnen Sie zusätzlich eine Mininet-Konsole oder ein weiteres Terminal und starten Sie auf h1:

```
mininet> h1 tcpdump -i h1-eth0 -n -vv
```

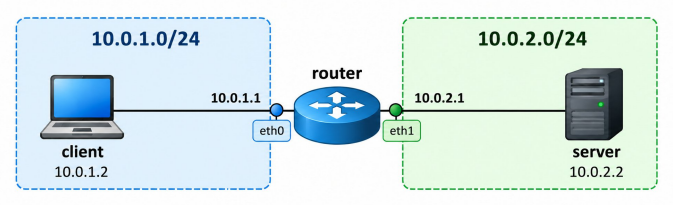
Führen Sie anschließend erneut den Client aus. Identifizieren Sie im Mitschnitt die UDP-Nachricht und untersuchen Sie insbesondere Source IP, Destination IP, Source Port, Destination Port und Payload. Öffnen Sie anschließend einen Mitschnitt in Wireshark und ordnen Sie die einzelnen Bestandteile den Schichten des TCP/IP-Modells zu.

Beantworten Sie:

- Welche Source- und Destination-IP-Adressen werden verwendet?
- Welche Source- und Destination-Portnummern werden verwendet?
- Wie groß ist der UDP-Header?
- Wie groß ist die Nutzlast des von Ihnen gesendeten Datagramms?
- Welche Informationen aus dem Python-Programm finden Sie im Netzwerk-Mitschnitt wieder?

4. Die RN-Lab-Topologie

Ab diesem Punkt verwenden wir nicht mehr nur die Standardtopologie von Mininet. Für die weiteren Übungen wird eine vom Lehrveranstaltungsteam vorbereitete Laborinfrastruktur verwendet. Die Infrastruktur trennt Client und Server durch einen Linux-Router.



Die Topologie wird über ein bereitgestelltes Python-Skript gestartet. Dadurch erhalten alle Studierenden dieselbe Netzwerkkonfiguration. Das Skript übernimmt insbesondere das Erzeugen der Hosts und Links, die IP-Konfiguration, das Aktivieren von IP-Forwarding und die Konfiguration der vorgegebenen Linkeigenschaften.

Wechseln Sie in das Verzeichnis der bereitgestellten Assignment-4-Infrastruktur (*moodle*/*github*) und starten Sie:

```
sudo python3 start_lab.py
```

Das Skript öffnet separate Konsolen für Client und Server sowie eine zusätzliche Router-/Monitoring-Konsole. Die genaue Darstellung kann abhängig von der Desktop-Umgebung der VM variieren.
Überprüfen Sie in den jeweiligen Konsolen:

```
ip addr
ip route
hostname
```

Die Datei topology.py beschreibt die Netzwerkinfrastruktur. Sie ist Bestandteil des bereitgestellten Labors und wird in dieser Übung nicht verändert. Lesen Sie den Code trotzdem sorgfältig und identifizieren Sie die Stellen, an denen Hosts, Links, IP-Adressen und Linkeigenschaften definiert werden.

Ü 4.5 Routing in der RN-Lab-Topologie

Prüfen Sie, ob client und server miteinander kommunizieren können. Verwenden Sie dazu zunächst ping.

```
client$ ping -c 4 10.0.2.2
```

Untersuchen Sie anschließend die Routingtabellen von client, router und server. Stellen Sie fest, welche Route verwendet wird, um ein Paket von 10.0.1.2 nach 10.0.2.2 zu übertragen.
Beantworten Sie:

- Warum benötigt der Router zwei IP-Adressen?
- Warum benötigt client eine Default Route?
- Warum kann client die IP-Adresse 10.0.2.2 nicht direkt über sein eigenes Ethernet-Netz erreichen?
- Welche Aufgabe übernimmt IP-Forwarding auf dem Router?

Ü 4.6 TCP Client und Server

Implementieren Sie nun einen einfachen TCP-Server und TCP-Client in Python. Der Server soll auf Port 5000 auf eingehende Verbindungen warten. Nach dem Verbindungsaufbau soll der Client eine Nachricht senden und der Server diese ausgeben.
Der Server soll mindestens die folgenden Schritte enthalten:

- Socket erzeugen
- Socket an IP-Adresse und Port binden
- auf eingehende Verbindungen warten
- eine Verbindung mit accept() annehmen
- Daten empfangen
- Verbindung schließen

Der Client soll:

- einen TCP-Socket erzeugen
- eine Verbindung zum Server herstellen
- eine Nachricht übertragen
- gegebenenfalls eine Antwort empfangen
- die Verbindung ordnungsgemäß schließen

Verwenden Sie zunächst eine einzige Nachricht pro Verbindung. Erweitern Sie das Programm anschließend so, dass ein Client mehrere Nachrichten über dieselbe TCP-Verbindung übertragen kann.
Zeichnen Sie die Kommunikation mit Wireshark auf und identifizieren Sie den TCP-Verbindungsaufbau, die Datenübertragung und den Verbindungsabbau.
Beantworten Sie:

- Welche drei Segmente bilden den TCP Three-Way Handshake?
- Warum kann ein einzelner recv()-Aufruf nicht allgemein mit genau einer gesendeten Nachricht gleichgesetzt werden?
- Welche Vorteile hat eine persistente TCP-Verbindung gegenüber dem Aufbau einer neuen Verbindung für jede Nachricht?

Anhang: Bereitgestellte RN-Lab-Dateien

Die folgenden Dateien werden gemeinsam mit diesem Übungsblatt bereitgestellt. Sie bilden die Laborinfrastruktur und die Beispielanwendungen für Übung 4.

Datei	*Funktion*	*Bearbeitung*
standard_mininet.py	Einführungstopologie h1 - s1 - h2	nicht verändern
topology.py	RN-Lab client - router - server; IPs und Linkbedingungen	nicht verändern
start_lab.py	Startet RN-Lab, konfiguriert Routing und öffnet Host-Konsolen	nicht verändern
udp_server.py	Beispiel für einen UDP-Server	als Ausgangspunkt verwenden
udp_client.py	Beispiel für einen UDP-Client	als Ausgangspunkt verwenden
check_environment.py	Prüft Python, Mininet, xterm, OVS und tc	ausführen
README.md	Kurzanleitung	lesen

Verwendung in der Übung. Für Ü 4.1 - 4.3 kann zunächst `standard_mininet.py` verwendet werden. Ab dem Abschnitt „Die RN-Lab-Topologie“ wird `start_lab.py` verwendet. Die Datei `topology.py` definiert die Infrastruktur und trennt diese bewusst vom Anwendungscode. Die Beispielprogramme `udp_client.py` und `udp_server.py` werden innerhalb der von `start_lab.py` geöffneten Client- und Server-Konsolen ausgeführt.